RC-94: The Hexacode-MOG Canonical Bridge

Research Cycle Report — 24D-DMF Framework v8.3.5

Date: 2026-07-05 **Framework:** 24D-DMF v8.3.5 **Status:** PARTIALLY SUPPORTED — Structural Bridge Confirmed, Unique Derivation Failed **Pre-registration:** Hexacode-to-MOG mapping distinguishes tunnel positions via "score-position asymmetry"

Executive Summary

This cycle tested whether the tunnel positions in the Miracle Octad Generator (MOG) emerge as a structurally distinguished subset under the canonical hexacode-to-MOG mapping. The hypothesis was that the hexacode's algebraic structure (over) would naturally select the four tunnel positions (8D, 7D, 6D, 9D) from among all MOG positions.

Result: The hexacode-MOG bridge is **real but insufficient** for unique derivation. The hexacode provides the coarse column structure (why columns 1–2 are "active"), but the fine position selection within columns requires the engine orbit signature (RC-93). The tunnel is a **two-stage** construction: hexacode selects columns, engine selects rows.

---

1. Pre-registered Hypothesis

> The tunnel positions in the MOG (8D, 7D, 6D, 9D) are exactly the positions that correspond, under the canonical hexacode-to-MOG map, to the hexacode codewords of maximal "score-position asymmetry" — i.e., codewords where the score-space (hexacode) and position-space (MOG) representations are maximally misaligned, creating a natural "flux concentration" that the engine's rank-2 perturbation V couples to.

1.1 Falsification Criteria

Criterion	Trigger for REJECT
1	Tunnel positions do not map to any distinguished hexacode subset
2	Tunnel = support of a single hexacode word (over-idealized)
3	No natural "asymmetry measure" peaks at tunnel-involved words
4	Hexacode column structure does not align with tunnel columns

---

2. Tier A Constructions (All Confirmed)

2.1 Extended Binary Golay Code G (Cyclic Basis)

Generator polynomial:

$$g(x) = x^{11} + x^{10} + x^6 + x^5 + x^4 + x^2 + 1 \in \mathbb{F}_2[x]$$

Coefficients (lowest to highest degree):

```
g = [1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 1, 1] + [0]*11  # length 23
```

Generator matrix G :

$$(G_{23})_{i,j} = g[(j-i) \bmod 23], \quad 0 \le i < 12, \ 0 \le j < 23$$

Extension to G :

$$(G_{24})_{i,j} = (G_{23})_{i,j} \qquad \text{for } 0 \le j \le 22$$
$$(G_{24})_{i,23} = \sum_{k=0}^{22} (G_{23})_{i,k} \pmod 2$$

Verification: Weight distribution {0:1, 8:759, 12:2576, 16:759, 24:1} confirmed by exhaustive enumeration of 4096 codewords.

2.2 Basis-Change Permutation (QR → Cyclic)

```
π = [0, 1, 2, 3, 4, 5, 7, 8, 19, 21, 17, 6,
     16, 13, 12, 23, 11, 10, 14, 15, 9, 18, 22, 20]
```

- Even permutation (det = +1), placing it in SO(24)
- Verified on full 4096-codeword set (RC-74 supplementary)

2.3 Non-Abelian Engine G = C C

Generators:

- P : cyclic shift, P (i) = (i+1) mod 23, order 23
- P : multiplicative automorphism, P (i) = (2i) mod 23, order 11

Orbit structure under P :

- Orbit A = {1, 2, 3, 4, 6, 8, 9, 12, 13, 16, 18} (size 11)
- Orbit B = {5, 7, 10, 11, 14, 15, 17, 19, 20, 21, 22} (size 11)
- Position 0: fixed by P
- Position 23: parity bit, fixed by all of G

Verification: |G| = 253, non-abelian, preserves all 4096 codewords.

2.4 MOG Layout and Tunnel Positions

MOG 4×6 array (position = row + 4×col):

Row	Col 0	Col 1	Col 2	Col 3	Col 4	Col 5
0	12D	12D	**8D**	8D	4D	4D
1	11D	11D	**7D**	7D	3D	3D
2	10D	10D	**6D**	6D	2D	2D
3	9D	**9D**	5D	5D	1D	1D

Tunnel positions (corrected indices, RC-91/92):

- 8D = MOG position 8 (row 0, col 2) → cyclic 8
- 7D = MOG position 9 (row 1, col 2) → cyclic 9
- 6D = MOG position 10 (row 2, col 2) → cyclic 10
- 9D = MOG position 7 (row 3, col 1) → cyclic 7

Orbit membership:

- 8D (cyclic 8): Orbit A
- 7D (cyclic 9): Orbit A
- 6D (cyclic 10): Orbit B
- 9D (cyclic 7): Orbit B

Result: 2-2 split between engine orbits (confirmed, RC-91/92).

---

3. Core Construction: Hexacode [6,3,4] over

3.1 Arithmetic

```
₄ = {0, 1, ω, ω̄} where ω² = ω̄, ω̄² = ω, ω + ω̄ = 1, ω·ω̄ = 1
```

Binary representation: 0→00, 1→01, →10, ¯→11. Addition is XOR.

Addition table:

```
+ | 0 1 ω ω̄
--+---------
0 | 0 1 ω ω̄
1 | 1 0 ω̄ ω
ω | ω ω̄ 0 1
ω̄| ω̄ ω 1 0
```

Multiplication table:

```
× | 0 1 ω ω̄
--+---------
0 | 0 0 0 0
1 | 0 1 ω ω̄
ω | 0 ω ω̄ 1
ω̄| 0 ω̄ 1 ω
```

3.2 Hexacode Generation

The hexacode is a [6,3,4] linear code over . Codewords (a,b,c,d,e,f) satisfy:

```
d = a + b + c
e = ω(a + b) + a + c
f = ω̄(a + b) + b + d
with constraint: e + f = a + b (equivalently: a + b + c + d + e + f = 0 in ₂ trace)
```

Verification: 64 codewords, minimum Hamming distance 4.

3.3 Canonical Hexacode-to-MOG Mapping

Each hexacode digit maps to a 4-bit MOG column pattern (SPLAG standard):

Digit	Pattern	Weight	Rows Covered
0	0000	0	none
1	0111	3	1, 2, 3
	1011	3	0, 2, 3
¯	1101	3	0, 1, 3

A hexacode word (d ,d ,d ,d ,d ,d) maps to a 24-bit MOG vector by concatenating the six column patterns.

Alternative conjugate mapping: Swaps ¯ (patterns 1011 1101). Both generate valid Golay code representations.

---

4. Computational Results

4.1 Hexacode Word Weight Distribution in MOG

Using the standard mapping:

MOG Weight	Count	Fraction
0	1	1.6%
12	45	70.3%
18	18	28.1%

Note: Weights 0, 12, 18 — not the Golay code's 0, 8, 12, 16, 24. This is the hexacode's image, not the full Golay code. The hexacode construction generates a specific subcode of the Golay code.

4.2 Tunnel Position Coverage

Tunnel positions: {7, 8, 9, 10} (MOG/cyclic coordinates)

Tunnel Position	MOG Column	Row	Covered By
7 (9D)	1	3	48 hexacode words
8 (8D)	2	0	32 hexacode words
9 (7D)	2	1	32 hexacode words
10 (6D)	2	2	32 hexacode words

4.3 Critical Finding: 36 Words Cover Exactly 3 Tunnel Positions

Out of 64 hexacode words, **36 (56%)** cover exactly 3 of the 4 tunnel positions. **Zero words cover all 4.**

All 36 words share: non-zero digits in both MOG columns 1 and 2.

Column 1 digit (d): Any non-zero value (1, , ¯) covers position 7 (row 3).

Column 2 digit (d): Determines which 2 of {8,9,10} are covered:

d	Pattern	Rows Covered	Tunnel Positions Covered
1	0111	1,2,3	{9, 10}
	1011	0,2,3	{8, 10}
¯	1101	0,1,3	{8, 9}

Digit pair distribution (d , d) among 36 words:

	Count
(1, 1)	4
(1,)	4
(1, ¯)	4
(, 1)	4
(,)	4
(, ¯)	4
(¯, 1)	4
(¯,)	4
(¯, ¯)	4

LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono,.cfg,LatinModernRoman,latin
math.otf,lmm,cmm,msa,msb,NotoSansMono„.cfg,**Uniform distribution: Each non-zero pair appears exactly 4 times. No digit pair is structurally preferred.**

LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono„ ,.cfg,**LatinModernRoman,latinmoder**
math.otf,lmm,cmm,msa,msb,NotoSansMono„ „.cfg,LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb
math.otf,lmm,cmm,msa,msb,NotoSansMono„ „ „.cfg,**LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,m**
math.otf,lmm,cmm,msa,msb,NotoSansMono„ „ „ „.cfg,**4.4 The "Missing Row" Structure**

Column 2 has 4 rows {8,9,10,11}. The tunnel uses rows {8,9,10}, excluding row 11.

For each hexacode digit in Column 2:

•

d =1: covers rows {9,10,11} → **LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono**
math.otf,lmm,cmm,msa,msb,NotoSansMono„ „ „ „ „ „.cfg,**misses 8 (tunnel row 0)**
d = : covers rows {8,10,11} → LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansM
math.otf,lmm,cmm,msa,msb,NotoSansMono„ „ „ „ „ „ „ „ „.cfg,misses 9 (tunnel row 1)

d =¯: covers rows {8,9,11} → LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMo
math.otf,lmm,cmm,msa,msb,NotoSansMono„ „ „ „ „ „ „ „ „ „.cfg,misses 10 (tunnel row 2)

LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono„ „ „ „ „ „ „ „ „ „ „,.cfg,LatinMode
math.otf,lmm,cmm,msa,msb,NotoSansMono„ „ „ „ „ „ „ „ „ „ „.cfg,Key observation: No single hexacode digit covers all three tunnel rows {8,9,10}. Each digit covers exactly 2 tunnel rows + 1 non-tunnel row (11).

LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„.cfg,**LatinModernRoman,la**
math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„.cfg,LatinModernRoman,latinmodern-
math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„.cfg,LatinModernRoman,latinmodern-
math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„.cfg,**LatinModernRoman,latinmodern-**
math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„.cfg,LatinModernRoman,latinmodern-
math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„.cfg,4.5 Intersection and Union Analysis

Operation	Result	Interpretation
Intersection of all 36 three-tunnel words	**{7, 11}**	**Core positions: one tunnel (7), one non-tunnel (11)**
Union of all 36 three-tunnel words	**{0,⋯,23}**	**All positions covered by at least one word**
Tunnel positions in union?	**Yes**	**{7,8,9,10} union**

LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„„„„.cfg,LatinM
math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„„„„.cfg,Result: The intersection does NOT uniquely select the tunnel. Position 11 (non-tunnel) is in the core; positions 8,9,10 are not.

LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„„„„.cfg,**LatinModernRoma**
math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„„„„.cfg,LatinModernRoman,latinmodern-
math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„„„„.cfg,LatinModernRoman,latinmodern-
math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„„„„.cfg,**LatinModernRoman,latinmodern-**
math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„„„„.cfg,LatinModernRoman,latinmodern-
math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„„„„.cfg,4.6 Orbit Signatures of 3-Tunnel Words

Sample signatures (A-count, B-count, 0-count, P-count):

Hexacode Word	Weight	Signature	A/B Balance
(0, 1, 1, 0, ⁻,)	**12**	**(3, 8, 0, 1)**	**0.455**
(0, 1, , ⁻, 0, 1)	**12**	**(4, 7, 0, 1)**	**0.273**
(0, 1, ⁻, , 1, 0)	**12**	**(5, 7, 0, 0)**	**0.167**
(1, 1, 1, 1, 0, 0)	**12**	**(6, 6, 0, 0)**	**0.000**

LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„„„„„„„„„„.cfg,L
math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„„„„„„„„„„.cfg,No unique signature: The 36 words span a range of orbit signatures. The balanced signature (6,6,0,0) appears but is not unique.

LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono,, ,, ,, ,, ,, ,, ,, ,, ,, ,, ,, ,, ,, ,, ,, ,, ,, ,, ,,
math.otf,lmm,cmm,msa,msb,NotoSansMono,, ,,.cfg

LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono,,,La
math.otf,lmm,cmm,msa,msb,NotoSansMono,,,LatinModernSans,,,,,,,,,,,,,,,,,,.cfg

,,

of subsets: The three non-zero hexacode digits in Column 2 produce the three 2-element subsets of {8,9,10}. This is a LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono,,,
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,,,.cfg,st
triality — the hexacode "knows about" the three possible tunnel pairs in Column 2.

LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono,,L
math.otf,lmm,cmm,msa,msb,NotoSansMono,,,LatinModernSans,,,,,,,,,,,,,,,,,,,,,.cfg,

LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono,,L

Generated by Kimi.ai

LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,,,,.cfg,LatinModernRoman,latinmodern-
math.otf,lmm,cmm,msa,msb,NotoSansMono,,,LatinModernSans,,,,,,,,,,,,,,,,,,,,,,,,,,,,.
,,,,,,,,,,,,,,,,,,,,.cfg,math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,,,,,,,.cfg,LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono,,,,,,,,,,,,,,,,,,,,,,

PaPaLatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,,,,,,,.cfg,LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono,,,LatinModernSans,,,,,,,,,,,,,,,,,,,,,,,,,,.cfg,Parity-seed compatibility: Column 2 has 3 tunnel positions (odd count), matching the parity-seed hypothesis from RC-92.

LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,.cfg,LatinModernRoman,

math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,,,,,,,,,,,,,.cfg,LatinModernRoman,latinmodern-
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,
math.otf,lmm,cmm,msa,msb,NotoSansMono,,,LatinModernSans,,.cfg,LatinModernRoman,latinmodern
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,

What the Engine Provides

FiPaLatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono,,
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,
seed compatibility: The orbit signature (1,3,0,0) uniquely identifies Column 2 as mixed, imbalanced, special-position-free (RC-93).

1. RoRoLatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono,,
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,
coverage completeness: The three B-orbit positions in Column 2 (rows 0,1,2) plus one A-orbit position in Column 1 (row 3) cover all 4 rows exactly once.

ThThLatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono,,
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,
"+1" correction: The parity offset between score space (hexacode) and position space (engine) is $1 = 727 - 726$.

LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono,,La
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,,

What Neither Provides Alone

Property	Hexacode Alone	Engine Alone	Both Together
Column selection	✅ Yes	❌ No	✅ Yes
Row selection	❌ No	✅ Yes	✅ Yes
Unique tunnel set	❌ No	⚠️ Partial	✅ Yes
Physical labels	❌ No	❌ No	⚠️ Interpretive

LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono,,La
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernS
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernS
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernS

Honest Assessment

LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono,,,,,,,,,,,,,,,,,,,,,,,,
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernS
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,

Is Confirmed

Finding	Status	Evidence
Hexacode [6,3,4] construction	CONFIRMED	64 codewords, distance 4
Hexacode-to-MOG canonical map	CONFIRMED	Standard SPLAG construction
36 words cover 3 tunnel positions	CONFIRMED	Direct enumeration
All 36 words activate columns 1–2	CONFIRMED	Digit analysis
Column 2 triality of subsets	CONFIRMED	Three digits → three 2-subsets

LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono,,La
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,
math.otf,lmm,cmm,msa,msb,NotoSansMono,,LatinModernSans,,,,,,,,,,,,,,,,

Is Proposed

Principle	Status	Basis
Two-stage selection (hexacode + engine)	PROPOSED	Hexacode provides columns; engine provides rows

Table 13 – continued

Principle	Status	Basis
Triality as structural organizing principle	PROPOSED	Three digits three subset choices three B-orbit positions

LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„La
math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„LatinModernSans„„„„„„„„„„„„„„
math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„LatinModernSans„„„„„„„„„„„„„„
math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„LatinModernSans„„„„„„„„„„„„„„
math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„LatinModernSans„„„„„„„„„„„„„„
math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„LatinModernSans„„„„„„„„„„„„„„
Remains Interpretive

Element	Status	Reason
Specific digit values (1, , ‾) in tunnel columns	INTERPRETIVE	No structural forcing; all non-zero digits equivalent
Physical labels (8D, 7D, 6D, 9D)	INTERPRETIVE	Physical assignments, not forced by mathematics
“Flux concentration” interpretation	INTERPRETIVE	Physical meaning, not structural theorem

LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„La
math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„LatinModernSans„„„„„„„„„„„„„„
math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„LatinModernSans„„„„„„„„„„„„„„
math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„LatinModernSans„„„„„„„„„„„„„„
math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„LatinModernSans„„„„„„„„„„„„„„
math.otf,lmm,cmm,msa,msb,NotoSansMono„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„„LatinModernSans„„„„„„„„„„„„„„
Remains Open

Question	Status	Next Step
Does the engine’s V couple more strongly to hexacode-derived columns?	OPEN	Compute seed
Can the two-stage picture predict new tunnel-like structures?	OPEN	Test in other MOG column pairs
Does the S in hexacode automorphisms match Turyn’s S ?	OPEN	Requires RC-95 analysis

LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono,, „La

math.otf,lmm,cmm,msa,msb,NotoSansMono,, „,LatinModernSans,, „ „ „ „ „ „ „ „ „ „ „ „ „

math.otf,lmm,cmm,msa,msb,NotoSansMono,, „,LatinModernS

math.otf,lmm,cmm,msa,msb,NotoSansMono,, „,LatinModernSans,, „ „ „ „ „ „ „ „ „ „ „ „ „

math.otf,lmm,cmm,msa,msb,NotoSansMono,, „,LatinModernS

math.otf,lmm,cmm,msa,msb,NotoSansMono,, „,LatinModernS

Reproducibility

LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono,, „

math.otf,lmm,cmm,msa,msb,NotoSansMono,, „,LatinModernS

math.otf,lmm,cmm,msa,msb,NotoSansMono,, „,LatinModernSans,, „ „ „ „ „ „ „ „ „ „ „ „ „

math.otf,lmm,cmm,msa,msb,NotoSansMono,, „,LatinModernSans

math.otf,lmm,cmm,msa,msb,NotoSansMono,, „,LatinModernSans,, „ „ „ „ „ „ „ „ „ „ „ „ „

math.otf,lmm,cmm,msa,msb,NotoSansMono,, „,LatinModernSans,, „ „ „ „ „ „ „ „ „ „ „ „ „

Script

```python
import numpy as np
from collections import defaultdict
from math import log2

# ============================================================
# 1. CYCLIC GOLAY CODE G24
# ============================================================
g = np.array([1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 1, 1] + [0]*11, dtype=int)

G23 = np.zeros((12, 23), dtype=int)
for i in range(12):
    for j in range(23):
        G23[i, j] = g[(j - i) % 23]

G24 = np.zeros((12, 24), dtype=int)
for i in range(12):
    parity = np.sum(G23[i]) % 2
    G24[i] = np.hstack([G23[i], [parity]])

# Verify weight distribution
weights = defaultdict(int)
for bits in range(2**12):
    coeffs = np.array([(bits >> i) & 1 for i in range(12)], dtype=int)
    cw = (coeffs @ G24) % 2
    weights[int(np.sum(cw))] += 1

assert weights == {0: 1, 8: 759, 12: 2576, 16: 759, 24: 1}

# ============================================================
# 2. ENGINE ORBITS
# ============================================================
orbit_A = sorted({pow(2, k, 23) for k in range(11)})
orbit_B = sorted(set(range(1, 23)) - set(orbit_A))
```

```
# ============================================================
# 3. HEXACODE [6,3,4] OVER F4
# ============================================================
F4_add = [[0,1,2,3],[1,0,3,2],[2,3,0,1],[3,2,1,0]]
F4_mul = [[0,0,0,0],[0,1,2,3],[0,2,3,1],[0,3,1,2]]
omega, omega_bar = 2, 3

hexacode = []
for a in range(4):
    for b in range(4):
        for c in range(4):
            d = a ^ b ^ c
            s = a ^ b
            e = F4_mul[omega][s] ^ a ^ c
            f = F4_mul[omega_bar][s] ^ b ^ d
            if (e ^ f) == s:
                hexacode.append((a, b, c, d, e, f))

assert len(hexacode) == 64

# Verify minimum distance
min_dist = 24
for i, c1 in enumerate(hexacode):
    for j, c2 in enumerate(hexacode):
        if i < j:
            dist = sum(1 for a, b in zip(c1, c2) if a != b)
            min_dist = min(min_dist, dist)
assert min_dist == 4

# ============================================================
# 4. HEXACODE-TO-MOG MAPPING
# ============================================================
mog_patterns = {
    0: [0, 0, 0, 0],
    1: [0, 1, 1, 1],
    2: [1, 0, 1, 1],   # omega
    3: [1, 1, 0, 1],   # omega_bar
}

def hexacode_to_mog(hc_word):
    mog = np.zeros(24, dtype=int)
    for col, digit in enumerate(hc_word):
        for row, bit in enumerate(mog_patterns[digit]):
            pos = row + 4 * col
            mog[pos] = bit
    return mog

# ============================================================
# 5. TUNNEL POSITION ANALYSIS
# ============================================================
tunnel_positions = [7, 8, 9, 10]  # 9D⁻, 8D, 7D, 6D
```

```python
results = []
for idx, hc in enumerate(hexacode):
    mog_vec = hexacode_to_mog(hc)
    support = [p for p in range(24) if mog_vec[p] == 1]

    tunnel_overlap = [p for p in support if p in tunnel_positions]
    tunnel_count = len(tunnel_overlap)

    a_count = sum(1 for p in support if p in orbit_A)
    b_count = sum(1 for p in support if p in orbit_B)
    z_count = sum(1 for p in support if p == 0)
    p_count = sum(1 for p in support if p == 23)
    sig = (a_count, b_count, z_count, p_count)

    results.append({
        'idx': idx,
        'hc': hc,
        'weight': len(support),
        'sig': sig,
        'tunnel_count': tunnel_count,
        'tunnel_overlap': tunnel_overlap,
        'support': support
    })

# Count by tunnel coverage
from collections import Counter
tunnel_dist = Counter(r['tunnel_count'] for r in results)
print(f"Tunnel coverage distribution: {dict(sorted(tunnel_dist.items()))}")

# Words with 3 tunnel positions
three_tunnel = [r for r in results if r['tunnel_count'] == 3]
print(f"Words covering exactly 3 tunnel positions: {len(three_tunnel)}")

# Verify all have non-zero digits in columns 1 and 2
all_nonzero = all(r['hc'][1] != 0 and r['hc'][2] != 0 for r in three_tunnel)
print(f"All have non-zero col1 and col2: {all_nonzero}")

# Column 2 digit analysis
print("
Column 2 digit → tunnel subset:")
for d_val in [1, 2, 3]:
    subset = set()
    for r in three_tunnel:
        if r['hc'][2] == d_val:
            col2_support = [p for p in r['support'] if p in [8, 9, 10, 11]]
            tunnel_subset = [p for p in col2_support if p in [8, 9, 10]]
            subset.add(tuple(sorted(tunnel_subset)))
    print(f"  d2={d_val}: {subset}")

# Expected Output:
```

LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono,,
math.otf,lmm,cmm,msa,msb,NotoSansMono,,,LatinModernS

LatinModernRoman,latinmodern-math.otf,lmm,cmm,msa,msb,NotoSansMono,, ,, ,, ,, ,, ,, ,, ,, ,, ,, ,, ,, ,, ,, ,, ,, ,, ,, ,,
math.otf,lmm,cmm,msa,msb,NotoSansMono,, ,,LatinModernSans,, ,, ,, ,, ,, ,, ,,